SAE 4.VCOD.01

SYSTEME D'INFORMATION DECISIONNEL



Djibril FOURNIER et Clément RINGEVAL

Décembre 2025

Introduction

Aujourd'hui, **l'ère numérique a beaucoup évolué** : un seul individu produit désormais des centaines de données par jour. Mais une problématique reste au centre de chaque entreprise : **comment stocker ces données et comment les rendre accessibles à tous ?**

C'est exactement à cette problématique que nous avons répondu. En effet, **posséder des données ne suffit pas** ; il faut savoir les rendre fiables et structurées dans le but d'aider à la prise de décision.

L'architecture mise en place dans ce projet repose sur la **suite Oracle**. Nous avons utilisé **Oracle XE** pour la base de données et **SQL Developer** pour l'interroger. Ensuite, nous avons utilisé **ODI** en tant qu'ETL (Extract Transform Load), afin de transformer les données et de les rendre utilisables.

Ce rapport présente notre mode de fonctionnement, notre démarche méthodologique, les solutions que nous avons mises en place, et surtout les résultats obtenus.

Sommaire

Pré-requis ---------- 04 à 06
Installation ---------- 07 à 09
Pré-création ---------- 10 à 12

Création :

- Base de donnée ---------- 13 à 14
- DSA ---------- 15 à 17
- ODS ---------- 18 à 19
- Data WareHouse ---------- 20 à 21

Conclusion ---------- 22
Bilan ---------- 23

Pré-requis

Avant d'entamer la phase de développement technique, nous avons consacré la première phase du projet à la structuration de notre collaboration.

Travaillant en binôme, il était impératif de définir une méthodologie claire : répartir les responsabilités, séquencer les tâches et estimer la charge de travail pour garantir le respect des échéances.

Dès les premières heures de travail, nous avons établi une feuille de route prévisionnelle (une sorte de ToDo List) afin de piloter l'avancement du projet.

Le tableau ci-dessous présente la répartition des tâches telle que nous l'avions envisagée au lancement du projet (Septembre 2025) :

Tâche	Estimation (Heures)	Responsable(s)	Date Limite
Création du fichier de suivis (celui-ci)	0h30	D.Fournier, C.Ringeval	Sep 5, 2025
Dictionnaire de données	2h30	D.Fournier, C.Ringeval	Sep 12, 2025
Modélisation (MCD, MLD, MPD)	3h00	Djibril FOURNIER	Sep 12, 2025
Script SQL : Création BDD	1h30	Djibril FOURNIER	Sep 19, 2025
Script SQL : Alimentation BDD	35h00	D.Fournier, C.Ringeval	Sep 19, 2025
Requêtes SQL (Q1-Q5)	1h00	Djibril FOURNIER	Sep 26, 2025
Requêtes SQL (Q6-Q10)	3h00	Clément Ringeval	Sep 26, 2025
Requêtes SQL (11-Q12)	5h30	D.Fournier, C.Ringeval	Sep 26, 2025
Conception Modèle en étoile	40h00	D.Fournier, C.Ringeval	Oct 3, 2025
Script SQL : Création entrepôt	15h00	D.Fournier, C.Ringeval	Oct 10, 2025
Script : Alimentation entrepôt	50h00	D.Fournier, C.Ringeval	Oct 17, 2025
Cahier de recette	5h00	D.Fournier, C.Ringeval	Oct 24, 2025
TOTAL ESTIME	**162h30**		

Pré-requis

Ce planning a servi de fil conducteur tout au long du projet. Cependant, avec le recul, nous pouvons noter plusieurs écarts entre cette vision théorique et la réalité du terrain :

- Lors de cette planification initiale, nous n'avions pas encore pleinement dimensionné l'importance des couches intermédiaires. Comme vous le constaterez, les étapes spécifiques au DSA (Data Staging Area) et à l'ODS (Operational Data Store) n'apparaissent pas explicitement dans ce tableau, alors qu'elles ont constitué une part majeure du travail effectif sous Oracle Data Integrator (ODI).

- S'agissant de notre première réalisation d'un entrepôt de données complet intégrant toute la chaîne décisionnelle, certaines estimations horaires se sont révélées surévaluées. La prise en main des outils et la gestion des cas particuliers dans les données n'ont pas nécessité autant de temps et d'adaptation que nous l’avions imaginé.

Cette planification nous a néanmoins permis de garder un cap et de nous assurer que la répartition de la charge de travail au sein du binôme restait équilibrée.
Nous avions cependant tenu à jour ce fichier. Voici comment il s’est transformé :

Pré-requis

Tâche	Estimation (Heures)	Responsable(s)	Commentaire	Date Limite
Création du fichier de suivis (celui-ci)	0h30	D.Fournier, C.Ringeval		Sep 5, 2025
Dictionnaire de données	2h30	D.Fournier, C.Ringeval	Quand l'un faisait la modélisation, l'autre faisait le dictionnaire de donnée	Sep 12, 2025
Modélisation (MCD, MLD, MPD)	2h30	D.Fournier, C.Ringeval		Sep 12, 2025
Script SQL : Création BDD + Alimentation	1h10	Djibril FOURNIER	Technique : Visual Studio Code et CTRL + F et le Regex	Oct 10, 2025
Requêtes SQL (Q1-Q12)	5h00	Clément RINGEVAL		Oct 10, 2025
Conception Modèle en étoile	1h00	D.Fournier, C.Ringeval	D.F. à commerncer, C.R à finis	Oct 24, 2025
Script SQL : Création entrepôt, DSA, ODS	2h00	Djibril FOURNIER		Oct 24, 2025
Connection de la base de donnée au DSA	6h00	D.Fournier, C.Ringeval	Fait via ODI - pas finis à temps	Nov 21, 2025
Connection du DSA à l'ODS	1h00	Clément RINGEVAL	Fait via ODI - retard rattrapé	Dec 5, 2025
Connection de l'ODS au DTW	3h00	D.Fournier, C.Ringeval	Fait via ODI	Jan 11, 2026
Création du rapport et du diaporama pour la présentation	8h00	Djibril FOURNIER		Jan 11, 2026
TOTAL ESTIME	**32h40**			

Installation

Cette phase a consisté à mettre en place l'architecture logicielle nécessaire au projet sur nos postes de travail.

Nous avons installé Oracle complétement. Chaque outil joue un rôle précis dans notre architecture :

- Oracle Database XE (Express Edition) : C'est le Système de Gestion de Base de Données (SGBD). Pour utiliser une analogie, c'est la "gare" ou l'entrepôt central où toutes les données sont physiquement stockées. Il agit comme le moteur arrière-plan.

- Oracle SQL Developer : C'est l'interface graphique (IDE) qui nous permet de dialoguer avec la base de données. Il agit comme un "traducteur" : il nous permet d'écrire des requêtes et de visualiser ce qui se trouve à l'intérieur d'Oracle XE.

- Oracle Data Integrator (ODI) : C'est notre outil d'ETL. C'est le "gestionnaire de flux" qui est chargé d'extraire les données, de les transformer selon nos règles métier et de les charger dans l'entrepôt.

Lors de la première tentative de connexion via SQL Developer et ODI, nous avons fait face à une impossibilité de joindre la base de données.

Le diagnostic :

- L'erreur retournée était TNS-12541 / TNS-12545 (Pas de listener actif).
- La commande lsnrctl status dans l'invite de commande confirmait qu'aucun listener n'écoutait le port.
- Dans les services Windows, nous avons constaté que si le service de la base de données (OracleServiceXE) était bien en cours d'exécution (RUNNING), le service du Listener associé à XE était absent. Un autre listener (lié à une installation d'Oracle 21c) existait mais était mal configuré.

Installation

Les causes identifiées :

- Absence de listener dédié XE : Oracle XE n’avait pas installé son propre service d'écoute.
- Conflit de configuration : Le fichier de configuration listener.ora faisait référence à un hôte (HOST=Cryptogramme.lan) que Windows ne parvenait pas à résoudre (DNS).
- SID Manquant : Le listener existant ne connaissait pas l'existence de la base "XE" (erreur TNS-01151).

La solution appliquée :
Nous avons procédé à une reconfiguration manuelle des fichiers réseaux d'Oracle :

- Modification du fichier listener.ora pour remplacer l'hôte par localhost (127.0.0.1) afin de contourner les problèmes de résolution DNS.
- Ajout manuel de l'entrée du SID XE dans le fichier pour forcer la reconnaissance de la base.
- Enregistrement de la base auprès du listener via SQLPlus :

```
sqlplus / as sysdba
ALTER SYSTEM REGISTER;
EXIT;
```

- Redémarrage du listener via les commandes :

```
lsnrctl stop OracleOraDB21Home1TNSListener
lsnrctl start OracleOraDB21Home1TNSListener
```

Commande réaliser dans un invite de commande en mode administrateur.

Résultat : La commande lsnrctl status a confirmé le succès de l'opération (Service "XE" has 1 instance).

Installation

Nous avons remarqué que les services Oracle (et particulièrement le Listener) consomment des ressources système importantes, même lorsque nous ne travaillons pas sur le projet, puisqu'il est constamment activé dés son installation.

Pour pallier ce problème, nous avons configuré les services Oracle en mode "Manuel" dans Windows. Afin de simplifier le lancement de l'environnement de travail sans avoir à démarrer chaque service à la main, nous avons créé un script Batch (.bat) exécuté en tant qu'administrateur via un raccourci dédié.

Ce script démarre le listener, puis lance les applications :
(Si on veut tout éteindre, on redémarre son ordinateur)

```
@echo off
:: Démarrage du Listener Oracle uniquement si nécessaire
net start OracleServiceXE
net start OracleOraDB21Home1TNSListener
net start OracleOraDB21Home1TNSListenerOracleOraDB21Home1TNSListener
net start OracleOraDB21Home1MTSRecoveryService

:: Lancement des outils
start "" "C:\sqldeveloper\sqldeveloper\sqldeveloper.exe"
start "" "C:\ODI\oracledi\client\odi.exe"
```

Cette optimisation nous a permis de garder un poste de travail fluide en dehors des heures de projet.

Une fois l'environnement stable, nous avons procédé aux configurations finales :

- Création des connexions SYSTEM dans SQL Developer et ODI (Superviseur).
- Création de l'environnement de travail dans ODI, nommé "SAE".
- Importation de la base de données source Chinook dans SQL Developer.
- Création du modèle de données correspondant dans ODI pour rendre les tables Chinook exploitables par l'ETL.

Pré-création

Avant d'entamer la modélisation décisionnelle, une phase d'information de la source de données était indispensable.
Notre objectif était de comprendre comment les données était ranger, et quelle données était intégré à cette base de donnée pour comprendre comment la base de donnée fonctionnais

La base de données source imposée pour ce projet est "Chinook".
Il s'agit d'une base de données relationnelle qui simule l'activité complète d'une boutique de musique en ligne (un modèle économique qui ressemble probablement à celui d'Apple Musique ou de Spotify).

L'analyse de la structure révèle que la base contient 11 tables interconnectées, couvrant trois périmètres fonctionnels :

- Le Catalogue Musical : Artist, Album, Track, Genre, MediaType, Playlist, PlaylistTrack
- La Gestion Commerciale & Clients : Customer, Invoice, InvoiceLine.
- La Gestion salariale : Employee

Cette structure relationnelle est probablement une structure idéale pour une application d'achat et d'écoute de musique en ligne. Mais, elle nécessite une transformation majeure pour être adaptée à l'analyse décisionnelle.

Pour faciliter le travail de modélisation futur (et notamment le passage au modèle en étoile : modèle le plus adapté avec celui en flocon pour la création d'outil d'aide à la prise de décision), nous avons réalisé un dictionnaire de données exhaustif. Ce document nous a permis de recenser chaque champ, son type, et surtout ses règles de gestion.

Pré-création

Voici un extrait du dictionnaire que nous avons constitué :

Code Rubrique	Nom Rubrique	Type	Règle d'intégrité
EmployeeId	Identifiant unique de l'employé	Numérique	Doit être unique indexé et non NULL
ReportsTo	Identifiant du supérieur hiérarchique	Numérique	Doit être une clé étrangère de EmployeeId, ou être NULL (pour le patron par exemple)
BirthDate	Date de naissance	Date	Doit être non NULL forcément antérieur à la date d'embauche
AlbumId	Identifiant unique de l'album	Numérique	Clé primaire, Unique et non NULL
Title	Titre de l'album	Texte	Non NULL

Note : Le dictionnaire complet est disponible en annexe de ce rapport.

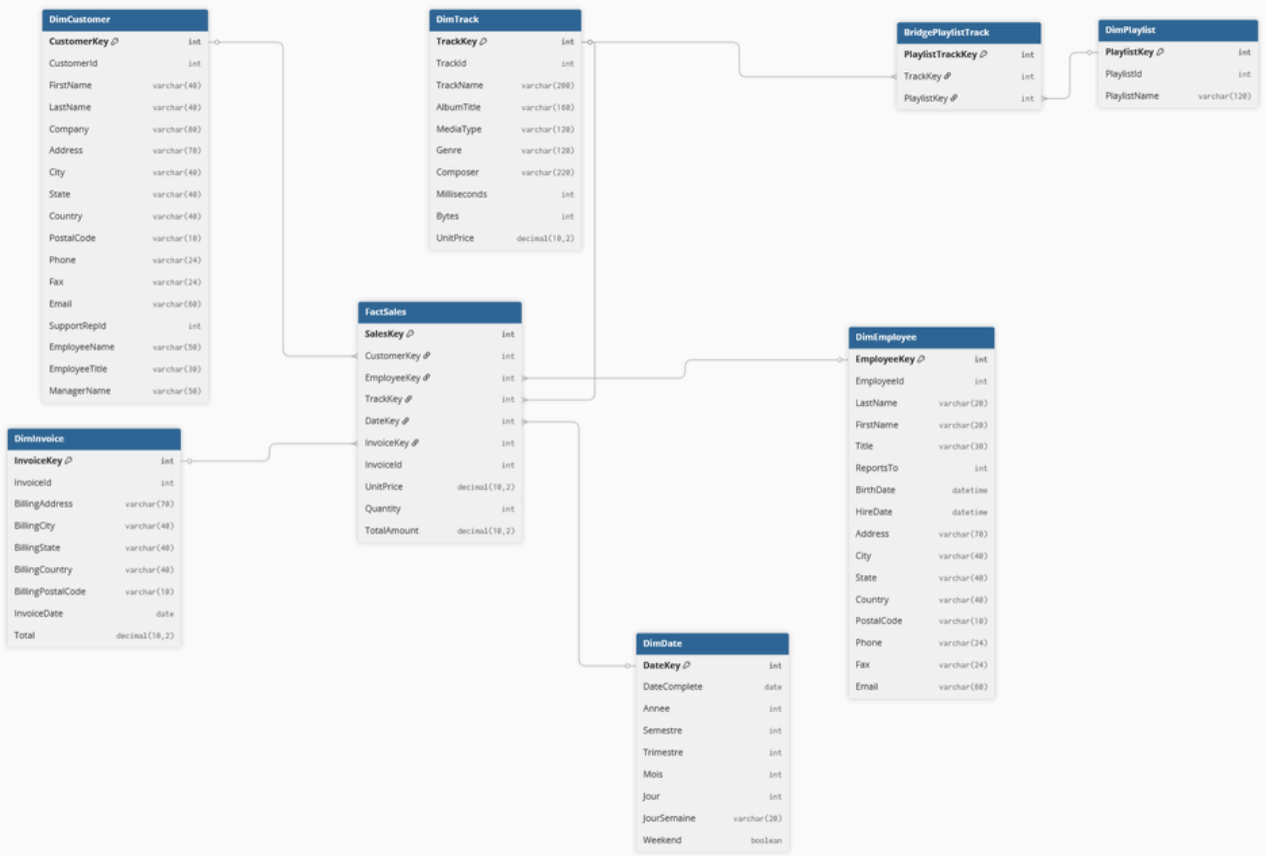
L'utilité de ce dictionnaire permet de mettre en valeur les champs essentiels et surtout les règles à respecter en cas de création de la base de donnée.

On retrouve donc des champs nécessaire au bon fonctionnement d'une base de donnée tel que le type; qui est très important car il faut par exemple éviter de stocker des dates dans un autre format que le type de date, les règles d'intégrité; car on souhaite que certain champs soit unique tel que les identifiants, puis le code rubrique; qui donne un nom et qu'on pourra utiliser pour retrouver le sens de cette variable dans ce dictionnaire de donnée.
Le nom rubrique à surtout un but indicatif, dans le cas ou un individu souhaite travailler sur une base de donnée sans même la connaître. Parfois, même la nomenclature de certain champs est très peu intuitif.

Pré-création

Cette étude préalable nous a permis de confirmer la faisabilité du projet et d'identifier les axes d'analyse possibles. Nous avons compris que pour créer notre Data WareHouse, nous allions devoir dénormaliser ces données pour passer d'une vision "Processus de vente" (base source) à une vision "Analyse des ventes" (Modèle en étoile).

Nous avons donc penser et conçu ce modèle en étoile. (Optimisé pour les SELECT et non les UPDATES/DELETE/INSERT.)



Dans ce modèle, on retrouve donc notre table de fait au centre. Et on remarque que notre modèle en étoile et plutôt un modèle en flocon en raison de la connexion existante entre la DimPlaylist et DimTrack (connecté ensemble via nôtre BridgePlaylistTrack).

Création : Base de donnée

Après avoir pris connaissance de la base de données, nous avons récupéré les scripts de création et d'alimentation. Une analyse initiale a révélé que le script source n'était pas adapté à la syntaxe Oracle.

Trois problèmes majeurs ont été identifiés et corrigés :

- **La syntaxe d'insertion :** Le script utilisait des commandes INSERT INTO groupées (plusieurs lignes de valeurs pour une seule commande), format non supporté nativement par Oracle pour ce type d'import. Il a fallu scinder ces commandes pour effectuer des insertions ligne par ligne.

- **Les délimiteurs de texte :** Le script utilisait des guillemets doubles (") pour les chaînes de caractères. Or, Oracle réserve ce signe aux identificateurs d'objets. Nous avons dû convertir l'ensemble des chaînes pour utiliser des guillemets simples ('), conformément au standard SQL Oracle.

- **La gestion des apostrophes internes :** Certaines données contenaient des apostrophes (ex: "90's"), ce qui créait des conflits d'interprétation.

Pour résoudre ces problèmes efficacement, nous avons exploité les fonctionnalités avancées de l'éditeur Visual Studio Code. Grâce à l'utilisation d'expressions régulières (Regex) via la fonction "Rechercher/Remplacer" (menu CTRL + F), nous avons pu reformater l'intégralité du code et doubler les apostrophes internes pour qu'elles soient correctement interprétées par Oracle.

Création : Base de donnée

Une fois le script transformer, nous avons procédé à la mise en place de l'architecture selon les étapes suivantes :

- Etape 1 : Préparation de la base de données (Oracle) : Création d'un nouvel utilisateur dédié et exécution du script corrigé pour générer les tables et insérer les données.

- Etape 2 : Installation du module de connaissance IKM. Le module visé est celui de SQL, il sert à définir comment ODI intègre les données dans la cible : stratégie de chargement, transformations, contrôles et gestion des erreurs.

- Etape 3 : Configuration de la Topologie (ODI) :
  - Définition du Schéma Physique : Création de la connexion vers la base de données Oracle (serveur, port, utilisateur).
  - Définition du Schéma Logique : Création d'une connexion liée au contexte de travail, qui sera ensuite mappée vers notre Schéma Physique. L’objectif est de lié, ODI à Oracle SQL Developer.

- Etape 4 : Modélisation : Création d'un nouveau Modèle dans le "Designer" ODI s'appuyant sur le Schéma Logique précédemment défini.

- Etape 5 : Reverse-Engineering : Lancement de l'ingénierie inverse (Reverse-Engineering) pour récupérer automatiquement les métadonnées (tables, colonnes, clés) depuis la base Oracle vers ODI car ce n’est pas une mise à jour automatique.

Cette procédure a permis de rendre les données pleinement exploitables pour les futurs flux d'intégration.

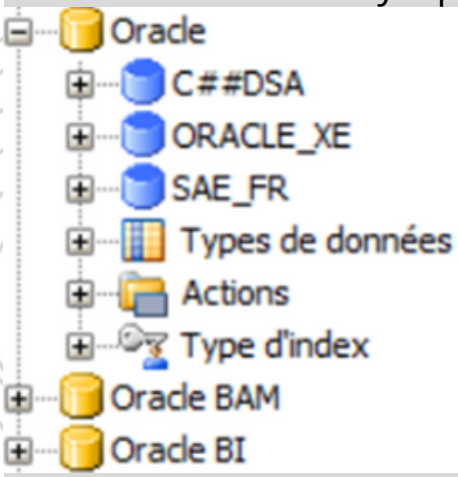
Création : DSA

Nous avons initié la construction de notre système décisionnel par la mise en place du DSA. Il a pour rôle de créer une copie des données sources de manière brute et identique, servant ainsi de zone “tampon” pour isoler les systèmes de production avant tout traitement complexe.

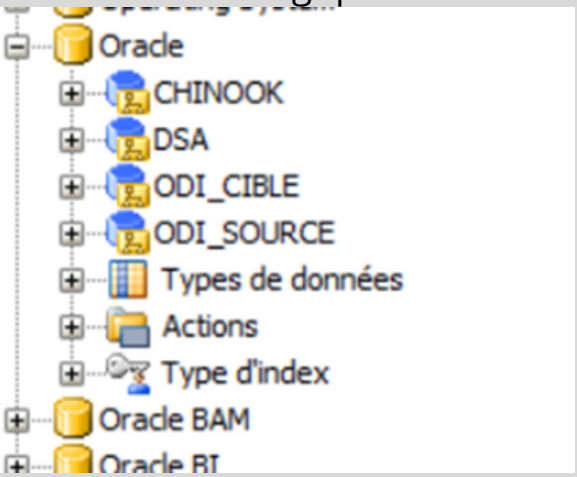
L'objectif technique était de répliquer les 11 tables de la base source Chinook vers notre schéma DSA, sans aucune transformation (pas de calculs, pas de filtres, etc...). Pour réaliser cette copie via Oracle Data Integrator (ODI), nous avons suivi une démarche structurée.

Avant de déplacer les données, nous avons défini nos structures dans le module "Designer" d'ODI. Nous avons effectué un Reverse Engineering sur la base source pour récupérer automatiquement les métadonnées (colonnes, types, clés primaires et étrangères) et nous avons créé un modèle physique et logique identique et vide pour le DSA, destiné à recevoir les données.

Modèle Physique :

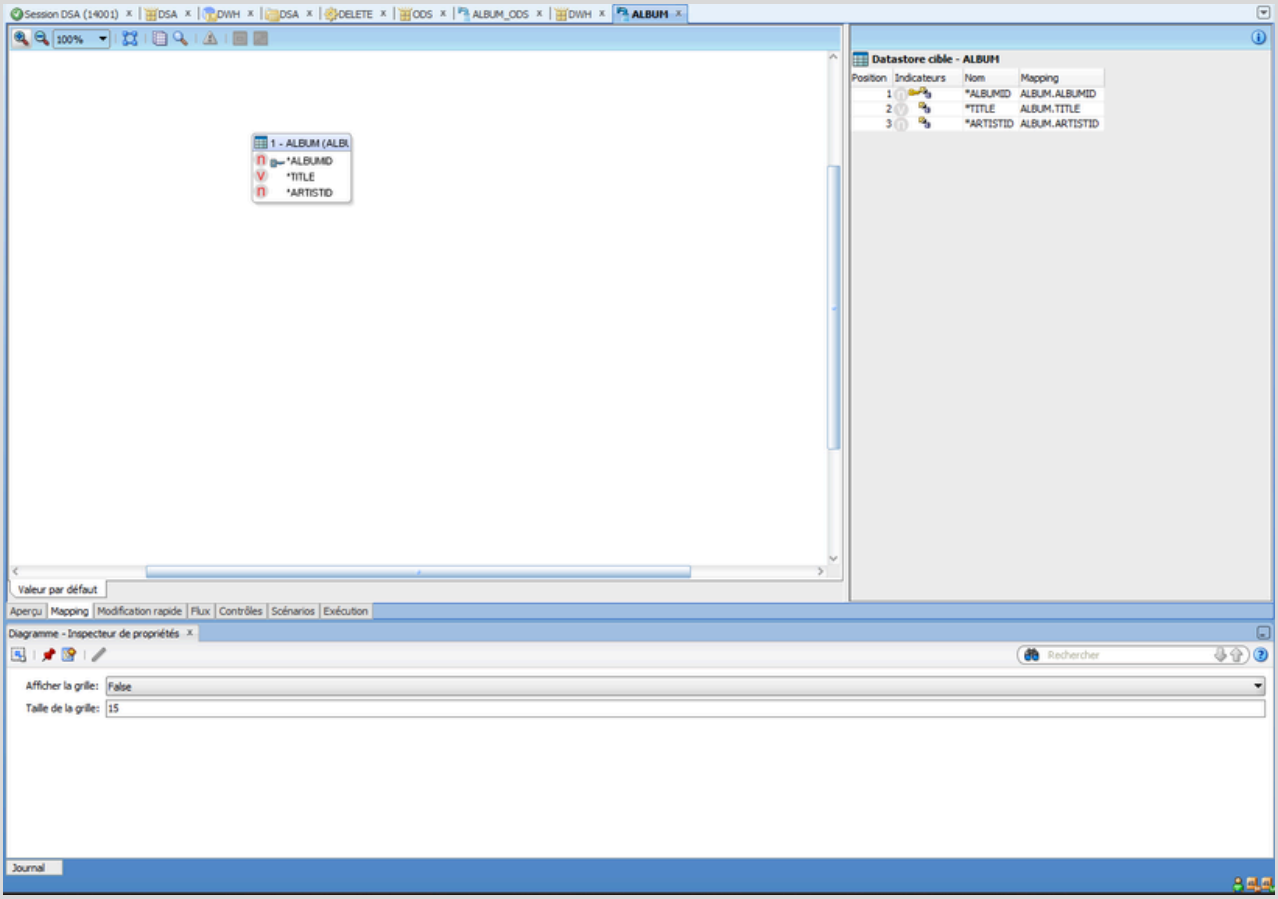


Modèle Logique :



Pour chaque table (ex: Album, Artist, Invoice...), nous avons ensuite créé une interface de flux de données (Mapping). Dans la Vue Logique, nous avons relié la table source à la table cible

Création : DSA



Sur la partie droite, on retrouve donc notre cible (notre table album du DSA), et sur la gauche, notre source (notre table album de Chinook)

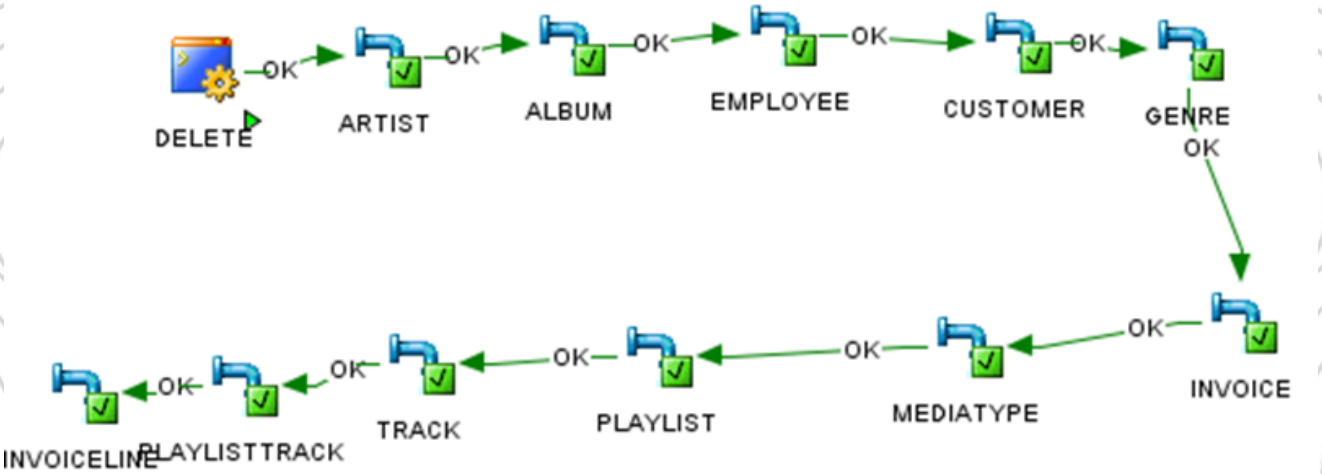
Lors de la configuration, nous avons rencontré une contrainte technique liée à l'intégrité référentielle. L'option standard Truncate Target Table (vider la table cible) provoquait des erreurs car on ne peut pas vider une table si une autre y fait référence. Pour contourner ce problème, nous avons désactivé cette option et développé une procédure spécifique qui vide les tables dans un ordre précis (en commençant par les tables "filles").

Ordre	Commande	Contexte	Schéma logique	Transaction	Valider (commit)	Ne pas prendre en c...	Commande fin
0	INVOICELINE		DSA			☐	☐
10	INVOICE		DSA			☐	☐
20	PLAYLISTTRACK		DSA			☐	☐
30	TRACK		DSA			☐	☐
40	GENRE		DSA			☐	☐
50	MEDIATYPE		DSA			☐	☐
60	ALBUM		DSA			☐	☐
70	ARTIST		DSA			☐	☐
80	PLAYLIST		DSA			☐	☐
90	CUSTOMER		DSA			☐	☐
100	EMPLOYEE		DSA			☐	☐

Création : DSA

Nous avons également activé le Flow Control pour vérifier que les données intégrées respectent bien les règles de typage.

Enfin, pour automatiser le tout, nous avons créé un Package global. Ce package agit comme un chef d'orchestre : il lance d'abord la procédure de nettoyage, puis exécute ensuite les 11 mappings séquentiellement. Cette automatisation nous permet, en un seul clic, de rafraîchir l'intégralité du DSA de manière cohérente.



Logiquement, on retrouve que la création est l'inverse de la suppression en raison des contraintes de clé primaire.

Création : ODS

Par la suite, nous avons fait l'ODS, L'ODS est la partie qui a pour but de consolider les données du DSA en y ajoutant une dimension temporelle (historisation par dates), ce qui permet de conserver une trace chronologique de l'évolution des données sans encore les transformer en modèles d'analyse.

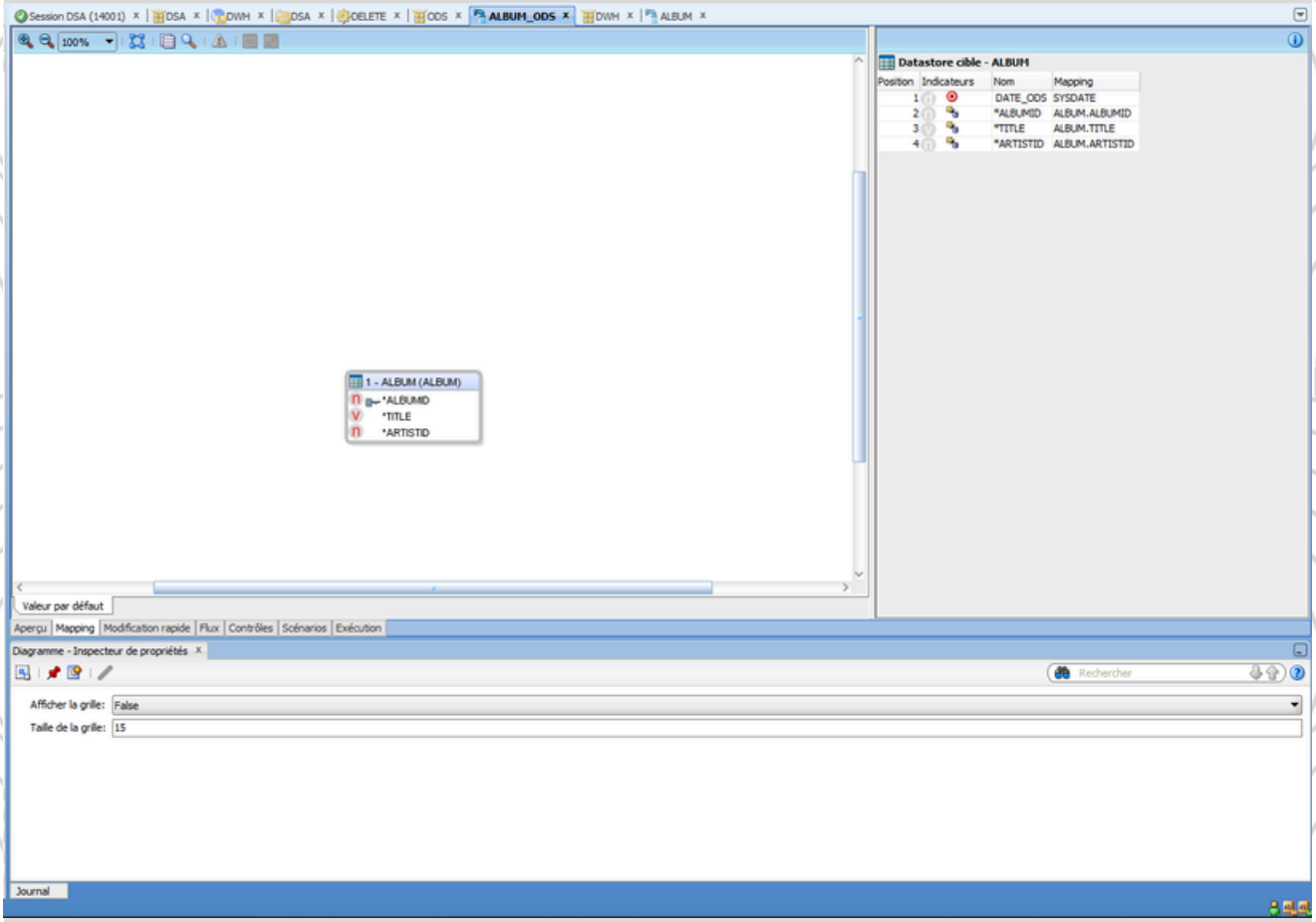
Contrairement au DSA qui est une image instantanée et volatile, l'ODS a pour but de consolider les données en y ajoutant une dimension temporelle. Cette étape est cruciale car elle permet de conserver une trace chronologique (historisation) de l'état des données à un instant T, avant leur transformation finale vers le modèle d'analyse.

La mise en œuvre a débuté par la préparation de la structure d'accueil dans la base de données. Nous avons rédigé et exécuté un script SQL spécifique pour créer les tables de l'ODS. Ces tables reprennent la structure exacte des données du DSA, mais avec une différence majeure : l'ajout systématique d'une colonne technique supplémentaire destinée à stocker la date et l'heure du chargement.

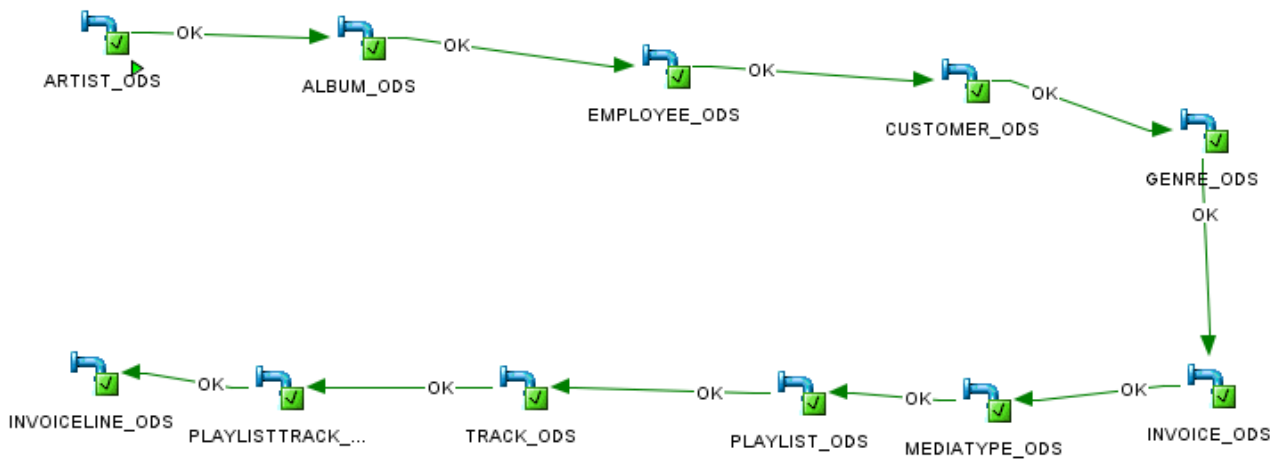
Une fois ces tables physiquement créées, nous avons effectué un Reverse Engineering dans ODI. Cela nous a permis d'importer ce nouveau modèle physique et logique, rendant les métadonnées de l'ODS disponibles pour nos développements.

Nous avons ensuite réalisé les mappings (flux de données). Ici, la source est le DSA et la cible est l'ODS. La particularité technique de cette étape réside dans le remplissage de la colonne supplémentaire : nous n'avons pas mappé de colonne source, mais nous avons utilisé une fonction système (SYSDATE) pour générer automatiquement la date et l'heure exactes au moment de l'insertion.

Création : ODS



Enfin, pour garantir la cohérence des données, nous avons automatisé le processus via une procédure globale. Celle-ci permet, en un seul clic, de déclencher l'alimentation séquentielle de toutes les tables de l'ODS. Cela assure que l'ensemble des données chargées lors d'une même exécution porte le même marqueur temporel.



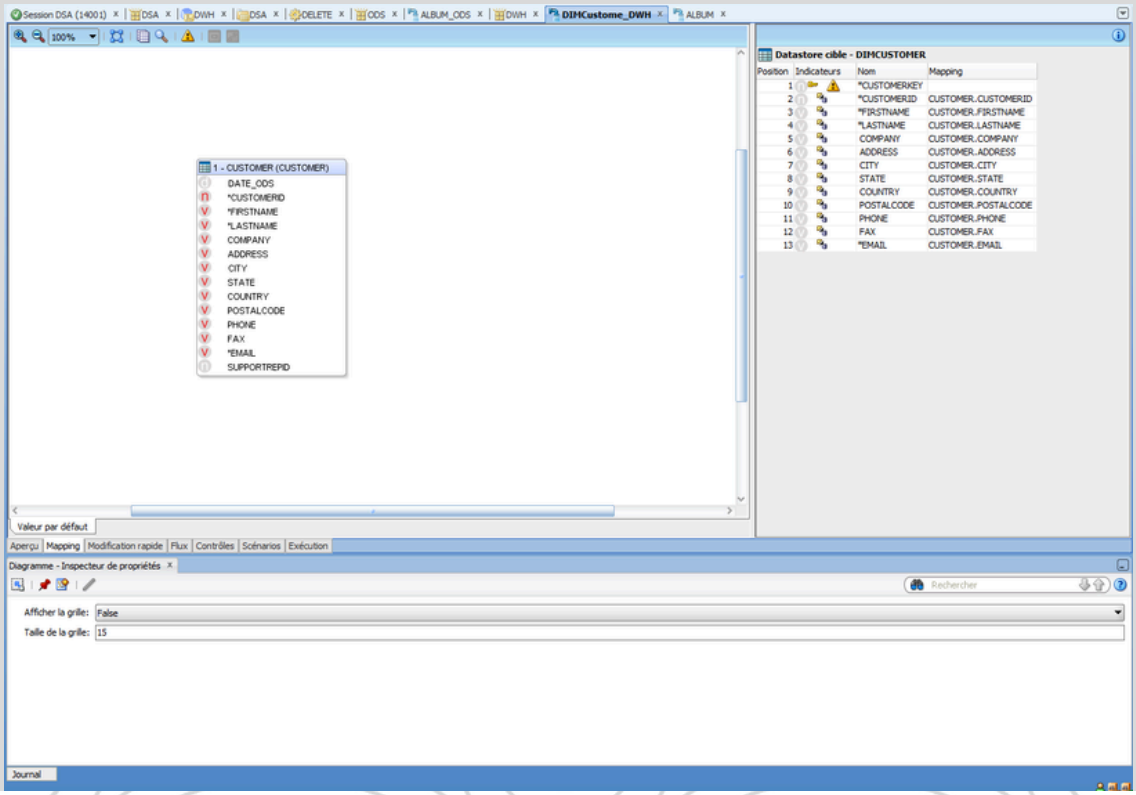
Création : Data WareHouse

Pour finir, la dernière étape est le Data WareHouse, autrement appelé DTW. Il est la partie qui a pour but de transformer et modéliser ces données historisées en schémas en étoile (tables de faits et dimensions précédemment présenter), afin de les rendre lisibles et optimisées pour les **rapports décisionnels**. C'est ici que nous passons d'un modèle relationnel classique à un modèle optimisé pour les requêtes décisionnelles.

La première étape a consisté à matérialiser notre modèle conceptuel (conçu en amont du projet) dans la base de données Oracle XE. Nous avons rédigé et exécuté un script SQL de création pour mettre en place les tables de notre modèle en étoile.

Une fois la structure créée en base, nous avons procédé à un Reverse Engineering dans ODI pour importer ce modèle physique et logique dans notre environnement de développement.

L'alimentation du DWH se fait depuis l'ODS. C'est lors de cette étape que les transformations métier s'opèrent. Nous avons créé les mappings pour alimenter les dimensions, puis pour la table de faits.



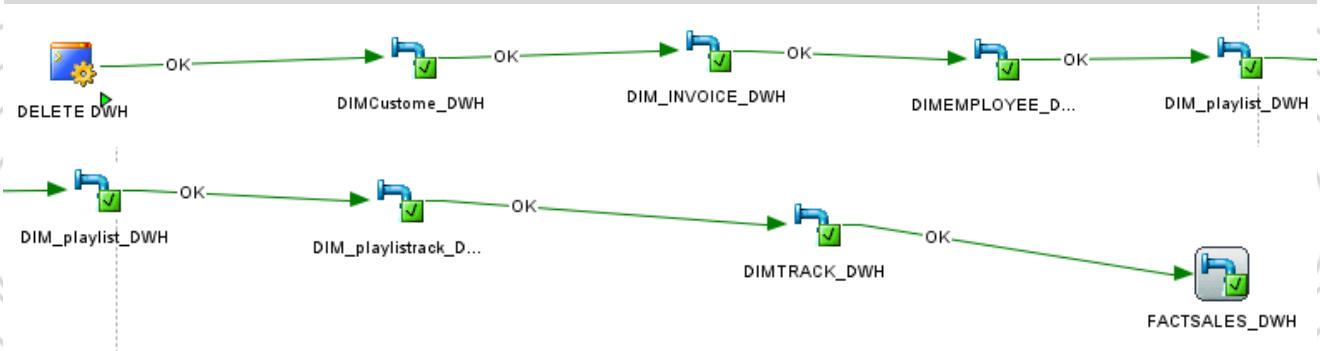
Création : Data WareHouse

Une spécificité technique importante a été mise en œuvre sur la Table de Faits : l'ajout d'une clé de substitution (ou Surrogate Key). Plutôt que de nous reposer uniquement sur les identifiants fonctionnels provenant de la source (qui peuvent changer ou être réutilisés), nous avons généré une clé technique unique interne au DWH. Cela garantit l'unicité absolue de chaque ligne de vente intégrée dans notre entrepôt.

Pour finaliser le projet, nous avons encapsulé l'intelligence du chargement dans une procédure d'automatisation. Cette procédure gère le cycle de vie complet du DWH lors d'un rafraîchissement :

- Suppression des données existantes (Remise à zéro).
- Alimentation des tables de Dimensions.
- Alimentation de la table de Faits (en respectant l'intégrité référentielle).

Grâce à cette automatisation, nous disposons désormais d'un système capable de transformer les données brutes de Chinook en indicateurs décisionnels fiables en quelques secondes.



Conclusion

Au terme de ce projet, nous sommes parvenus à répondre à la problématique initiale : transformer un flux de données brutes et hétérogènes en une source d'information fiable et structurée pour la prise de décision.

La réalisation de cette chaîne décisionnelle complète, basée sur l'écosystème Oracle (XE, SQL Developer, ODI), nous a permis de concrétiser les concepts théoriques de la Business Intelligence. En partant de la base de données transactionnelle Chinook, nous avons construit, étape par étape, une architecture robuste :

1. Sécurisation des données via le DSA (zone tampon).
2. Historisation et consolidation via l'ODS.
3. Restructuration pour l'analyse via le Data WareHouse et son modèle en étoile.

Au-delà de la simple exécution technique, ce projet a mis en lumière l'importance de l'automatisation. Grâce aux procédures et packages développés sous ODI, le processus d'alimentation est désormais industriel : un simple lancement suffit pour rafraîchir l'intégralité de l'entrepôt, garantissant ainsi des données toujours à jour.

Même en dehors d’Oracle, nous avons appris d’autre processus important tel que la création d’un modèle en étoile correcte.

Bilan

Ce travail nous a permis de toucher du doigt la réalité du métier d'Ingénieur Data. Nous avons compris que la valeur de la donnée ne réside pas uniquement dans son stockage, mais dans sa fiabilité et sa structure.
Nous avons assimilé l'importance de l'architecture en couches (DSA -> ODS -> DWH) :

- Isoler la source pour ne pas impacter la production (DSA).
- Historiser pour ne pas perdre d'information (ODS).
- Modéliser pour faciliter la prise de décision (DWH).

Travailler en binôme sur une architecture complexe a nécessité une rigueur organisationnelle accrue.

- Planification : L'élaboration et le suivi de notre planning (ToDoList) nous ont appris à estimer nos charges de travail, parfois en les surestimant, parfois en les sous-estimant face aux imprévus techniques.

- Résolution de problèmes : Nous avons appris à ne pas baisser les bras face aux erreurs techniques (problèmes de Listener TNS, configuration des services). Cette persévérance pour déboguer notre environnement est sans doute la compétence la plus précieuse acquise durant ce module.

Pour finir, nous avons appris beaucoup de choses nouvelles sur la suite Oracle.

En conclusion, ce projet ne s'est pas limité à "créer une base de données", mais à construire un système d'information complet, robuste et automatisé, confirmant ainsi notre intérêt pour le domaine de la Business Intelligence.